

Breve Introducción al lenguaje SQL

Diego Parrás



Breve Introducción al lenguaje SQL

Diego Parrás



VERSION DE LIBRE DISTRIBUCIÓN. PROHIBIDA SU COMERCIALIZACIÓN.



Contenido

1.	Antes de comenzar	6
2.	Bases de Datos Relacionales	11
3.	Principios del lenguaje SQL	12
3.1	Lenguaje de Definición de Datos (DDL)	12
2.1.1.	Tipos de Datos en SQL	13
2.1.2.	Clave Primaria (Primary Key) y Clave Foránea (Foreing Key)	14
3.2	Lenguaje de Manipulación de Datos (DML)	16
2.2.1.	Cláusulas y Operadores en SQL	17
2.2.1.1.	Cláusulas en SQL	17
2.2.1.2.	Operadores en SQL	20
2.2.1.2.1.	Operadores Aritméticos	20
2.2.1.2.2.	Operadores de Comparación	21
2.2.1.2.3.	Operadores Lógicos	22
2.2.1.2.4.	Operadores de Conjunto	23
2.2.1.2.5.	Operadores de Concatenación	24
3.3	Lenguaje de Consulta de Datos (DQL)	24
3.4	Lenguaje de Control de Datos (DCL)	24
3.5	Lenguaje de Control de Transacciones (TCL)	25
4.	Funciones en SQL	26
4.1	Funciones escalares	26
4.2	Funciones de conjunto	28
4.3	Implementación de Funciones en Consultas SQL	29
5.	Store Procedures	29
6.	Triggers en SQL	31
7.	Estructuras de Control en SQL	33
8.	Vistas en SQL	36
8.1	Ventajas de las vistas en SQL	36
8.2	Sintaxis en la creación de vistas	37
8.3	Tipos de vistas en SQL	37
8.4	Vistas materializadas	38
9.	Subconsultas en SQL	39
10.	Ejercicios Resueltos	43
10.1	Creación de Bases de Datos y Tablas en SQL	43
10.2	Select, From, Where, Order By	44

10.3	Group By, Having, Count, Avg, Sum, Min, Max.....	45
10.4	Join, Inner Join, Left Join, Right Join, Full Join, Cross Join	46
10.5	Union, Intersect, Except	47
10.6	Subconsultas, Any, All, Exists	48
10.7	Crear, Alterar, Eliminar Tablas.....	49
10.8	Insert, Update, Delete	50
10.9	Funciones, Alias, Concatenación	51
11.	Conclusión.....	53

Breve Introducción al lenguaje SQL.

1. Antes de comenzar.

En nuestra era actual, caracterizada por la explosión de datos, el Lenguaje de Consulta Estructurada, o SQL, ha tomado una importancia vital en una variedad de campos. Como principal herramienta utilizada para interactuar con las bases de datos, SQL es una habilidad esencial para cualquiera que busque analizar, manipular y comprender datos. Este trabajo, "Breve Introducción al Lenguaje SQL", proporciona una visión compacta pero comprensiva de los aspectos más fundamentales de SQL.

Comenzaremos con una introducción a las Bases de Datos Relacionales, proporcionando el contexto necesario para entender cómo y por qué utilizamos SQL. A continuación, exploraremos los principios básicos de SQL, abordando el Lenguaje de Definición de Datos (DDL) así como también el Lenguaje de Manipulación de Datos (DML), sin olvidarnos de los demás.

En secciones posteriores, abordamos conceptos esenciales tales como las funciones en SQL, los procedimientos almacenados y los triggers, asegurándonos de que, aunque este sea un recorrido breve, los lectores tengan una visión completa de las herramientas y técnicas fundamentales que SQL ofrece.

Concluimos con una mirada a aspectos más avanzados de SQL, incluyendo las estructuras de control, las vistas y las subconsultas. Aunque estos temas pueden ser complejos, nos esforzamos por presentarlos de manera accesible y fácil de entender, de modo que incluso los principiantes puedan empezar a explorar estos conceptos más avanzados.

Desde el Centro de Estudios en Modelos de Inteligencia Artificial aplicados a las Ciencias Económicas de la Universidad de Buenos Aires (CeMIACE) hemos preparado este material, con la expectativa de que sea un recurso accesible y manejable para cualquier persona que desee obtener una visión general sólida de SQL. Aunque breve, esperamos que este trabajo proporcione a los lectores las bases que necesitan para sentirse cómodos y seguros al trabajar con SQL.

Luego de la parte teórico conceptual, ponemos a disposición una guía resuelta de ejercicios en SQL, para que todos los lectores que así lo quieran, puedan practicar este maravilloso lenguaje.

Lic. Diego Parrás (Director del CeMIACE)

BLOQUE TEÓRICO CONCEPTUAL

2. Bases de Datos Relacionales

Una base de datos relacional es un tipo de base de datos que almacena y proporciona acceso a datos relacionados entre sí. En este tipo de bases de datos, los datos se organizan en tablas, que a su vez se componen de filas y columnas. Cada fila representa un registro único y cada columna representa un campo o una dimensión de datos.

Las tablas se pueden vincular o relacionar entre sí a través de claves, que son identificadores únicos para cada registro. Por ejemplo, en una base de datos de una tienda en línea, cada cliente podría tener un ID de cliente único, cada producto podría tener un ID de producto único, y cada pedido podría tener un ID de pedido único. Estas claves permiten relaciones entre las tablas. Por ejemplo, un pedido puede estar vinculado a un cliente a través del ID del cliente, y puede contener varios productos a través de los ID de los productos.

Las bases de datos relacionales permiten realizar consultas complejas y manipulación de datos. Por ejemplo, podríamos querer buscar todos los pedidos realizados por un cliente específico, o todos los productos que se han pedido más de x cantidad de veces, o el cliente que ha gastado más dinero en total, etc. Todo esto se puede hacer mediante el uso de un lenguaje de consulta de base de datos como **SQL (Structured Query Language)**. Algunos ejemplos de sistemas de gestión de bases de datos relacionales son **MySQL**, **PostgreSQL**, **Oracle Database**, **Microsoft SQL Server**, entre otros.

Las bases de datos relacionales tienen una serie de ventajas, como la eficiencia, la flexibilidad y la capacidad de manejar grandes volúmenes de datos, así como soporte para operaciones de transacción y control de concurrencia. Sin embargo, también tienen algunas limitaciones, como la dificultad para manejar datos no estructurados y la necesidad de un diseño cuidadoso para garantizar un rendimiento óptimo. Por ejemplo, podríamos tener una base de datos para una tienda en línea que tenga una tabla **Productos**, una tabla **Clientes** y una tabla **Pedidos**, donde cada pedido podría estar relacionado con un cliente específico y contener uno o varios productos. La tabla **Clientes** podría tener columnas como **ID_cliente**, **nombre**, **direccion**, etc. La tabla **Productos** podría tener columnas como **ID_producto**, **descripcion**, **precio**, etc. La tabla **Pedidos** podría tener columnas como **ID_pedido**, **ID_cliente**, **fecha_pedido**, etc. Además, podría haber una tabla adicional, como **Detalles_pedidos**, que contenga **ID_pedido**, **ID_producto**, **cantidad**, etc. para representar los productos en cada pedido.

3. Principios del lenguaje SQL

El lenguaje **SQL** es una herramienta poderosa y esencial en el mundo de la tecnología de la información. Se utiliza principalmente para interactuar con bases de datos, permitiendo a los usuarios manipular y extraer información de ellas. Aunque a primera vista puede parecer una tecnología compleja, entender los fundamentos de **SQL** puede simplificar enormemente su aprendizaje y uso.

SQL se divide en cinco subconjuntos principales, cada uno con un propósito específico. Estos subconjuntos son el Lenguaje de Definición de Datos (**DDL**), el Lenguaje de Manipulación de Datos (**DML**), el Lenguaje de Consulta de Datos (**DQL**), el Lenguaje de Control de Datos (**DCL**) y el Lenguaje de Control de Transacciones (**TCL**). Cada uno de estos componentes de **SQL** proporciona una serie de comandos que permiten a los usuarios definir estructuras de datos, manipular datos, realizar consultas de datos y controlar el acceso a los datos.

3.1 Lenguaje de Definición de Datos (DDL)

Data Definition Language (DDL), Como su nombre indica, este subconjunto de **SQL** se encarga de la definición y la modificación de la estructura de los objetos de la base de datos. **DDL** permite crear, alterar y eliminar estructuras de base de datos, pero no manipula los datos dentro de estas estructuras. Esencialmente, los comandos **DDL** definen "qué" datos se almacenarán y "cómo" se organizarán. Los comandos **DDL** más utilizados son:

CREATE. Este comando se utiliza para crear nuevas bases de datos, tablas e índices. Por ejemplo, un comando **CREATE** para una tabla podría verse así:

```
CREATE DATABASE Prueba;
```

En el ejemplo anterior, se creó la base de datos **Prueba**.

```
CREATE TABLE Customers (  
  CustomerID INT PRIMARY KEY,  
  FirstName VARCHAR(100),  
  LastName VARCHAR(100),  
  Email VARCHAR(255)  
);
```

En el ejemplo, se creó la tabla **Customers** con las columnas **CustomerID**, **FirstName**, **LastName** y **Email**.

Es importante remarcar que con el comando **CREATE** se definen los tipos de datos correspondientes a cada campo, así como también la existencia de llaves primarias y/o foráneas.

ALTER. Este comando se utiliza para modificar una estructura de base de datos existente. Podemos utilizar **ALTER** para agregar o eliminar columnas de una tabla, cambiar el tipo de datos de una columna, modificar el tamaño de una columna, etc. Por ejemplo, para agregar una nueva columna a una tabla existente, podríamos usar este comando:

```
ALTER TABLE Customers ADD PhoneNumber VARCHAR(20);
```

En este ejemplo, se está agregando una nueva columna llamada **PhoneNumber** a la tabla **Customers**.

DROP. Este comando se utiliza para eliminar completamente una estructura de base de datos, como una tabla o una base de datos entera. Por ejemplo:

```
DROP TABLE Customers;
```

En este caso, la tabla **Customers** será eliminada de la base de datos, junto con todos los datos que contiene. Debemos tener cuidado con este comando, porque una vez que se elimina una tabla, no podremos recuperar los datos posteriormente.

TRUNCATE. Este comando se utiliza para eliminar todos los registros de una tabla, pero a diferencia del comando **DROP**, **TRUNCATE** no elimina la tabla en sí. La estructura de la tabla permanece, pero todos los datos se borran. Por ejemplo:

```
TRUNCATE TABLE Customers;
```

Este comando borrará todos los registros de la tabla **Customers**, pero la tabla **Customers** seguirá existiendo en la base de datos.

Es importante destacar que los comandos **DDL** son irreversibles, lo que significa que una vez que se ejecutan, no podremos deshacer sus efectos. Por lo tanto, es importante tener mucho cuidado al usar estos comandos, especialmente en un entorno de producción.

2.1.1. Tipos de Datos en SQL

Al diseñar las tablas para una base de datos en **SQL**, uno de los aspectos más críticos a considerar es el tipo de datos que se utilizará para cada campo. Los tipos de datos son fundamentales porque determinan qué tipo de información puede almacenarse en un campo

específico, cómo **SQL** interactuará con esa información y cuánto espacio ocupará en la memoria. **SQL** soporta una amplia variedad de tipos de datos, que se pueden clasificar en varias categorías principales: numéricos, de fecha y hora, de cadena de caracteres, binarios y otros.

Tipos de datos numéricos. Estos incluyen enteros (**INT**), números decimales y de punto flotante (**DECIMAL**, **FLOAT**), entre otros. Por ejemplo, puedes usar **INT** para almacenar valores enteros, y **DECIMAL** para almacenar números con decimales.

Tipos de datos de fecha y hora. Estos permiten almacenar valores de fecha y hora. Algunos ejemplos son **DATE**, **TIME** y **DATETIME**.

Tipos de datos de cadena de caracteres. Estos se utilizan para almacenar texto. **VARCHAR** y **CHAR** son dos ejemplos comunes de este tipo de datos. **VARCHAR** es para cadenas de longitud variable, mientras que **CHAR** es para cadenas de longitud fija.

Tipos de datos binarios. Estos se utilizan para almacenar datos que no están destinados a ser leídos como texto. Algunos ejemplos son **BINARY** y **VARBINARY**.

Otros tipos de datos. SQL también admite tipos de datos especializados como **BOOLEAN** (para verdadero/falso), **ENUM** (para un valor seleccionado de una lista predefinida), y otros.

Resulta importante recordar que el soporte para tipos de datos específicos puede variar entre los diferentes sistemas de gestión de bases de datos (**DBMS**), aunque la mayoría de los tipos mencionados son estándares y ampliamente soportados.

2.1.2. Clave Primaria (Primary Key) y Clave Foránea (Foreign Key)

En el diseño de bases de datos relacionales, una **clave primaria** es un elemento fundamental que sirve como identificador único de cada fila en una tabla. Este identificador único puede estar compuesto por un solo campo o atributo (por ejemplo, un ID de usuario o un número de serie), o puede ser una combinación de múltiples campos.

Lo que define y diferencia a una clave primaria es que cada valor debe ser único y no puede ser nulo. Esto significa que no podremos tener dos filas en la misma tabla con el mismo valor de clave primaria, y cada fila debe tener un valor de clave primaria. Estas restricciones garantizan la identidad única de cada fila y permiten una rápida búsqueda y acceso a los datos.

En esencia, la clave primaria de una tabla es la columna o el conjunto de columnas utilizado para diferenciar un registro de todos los demás en la tabla. Cada tabla en una base de

datos relacional debe tener una y solo una clave primaria. Esta clave se utiliza no sólo para garantizar que los datos en una tabla sean únicos, sino también para establecer relaciones con otras tablas. Los valores de la clave primaria de una tabla se pueden usar en otras tablas como claves foráneas para crear vínculos entre las filas de diferentes tablas.

Por ejemplo, en la siguiente tabla **Cientes**, la columna **ID_Cliente** es la clave primaria:

```
CREATE TABLE Cientes (  
  ID_Cliente INT PRIMARY KEY,  
  Nombre VARCHAR(100),  
  Email VARCHAR(100)  
);  
INSERT INTO Cientes VALUES (1, 'Juan', 'juan@example.com');  
INSERT INTO Cientes VALUES (2, 'Ana', 'ana@example.com');  
INSERT INTO Cientes VALUES (3, 'Carlos',  
'carlos@example.com');
```

*En el ejemplo, cada cliente tiene un **ID_Cliente** único.*

Una **clave foránea** es un campo o un conjunto de campos que proporciona un enlace entre los datos de dos tablas. Es la columna en una tabla que es la clave primaria en otra tabla. La clave foránea se utiliza para prevenir acciones que eliminarían enlaces entre las tablas.

Un registro en una tabla que tiene una clave foránea debe tener un valor correspondiente en la tabla principal. Por ejemplo, podemos tener otra tabla **Pedidos** que tiene una clave foránea **ID_Cliente**:

```
CREATE TABLE Pedidos (  
  ID_Pedido INT PRIMARY KEY,  
  ID_Cliente INT,  
  Fecha_Pedido DATE,  
  Total DECIMAL(8, 2),  
  FOREIGN KEY (ID_Cliente) REFERENCES Cientes(ID_Cliente)  
);  
  
INSERT INTO Pedidos VALUES (1, 1, '2023-07-20', 150.00);  
INSERT INTO Pedidos VALUES (2, 2, '2023-07-21', 200.00);
```

En esta tabla, **ID_Cliente** es una clave foránea que vincula cada pedido con el cliente que lo realizó. Cada **ID_Cliente** en la tabla **Pedidos** debe coincidir con un **ID_Cliente** en la tabla **Cientes**. Esto garantiza la integridad de los datos y establece la relación entre las dos tablas.

3.2 Lenguaje de Manipulación de Datos (DML)

Data Manipulation Language (DML) es la parte de **SQL** que nos permite manipular y modificar los datos almacenados en nuestra base de datos. Aquí es donde se realizan todas las operaciones de inserción, consulta, actualización y eliminación. A continuación, vamos a profundizar en los comandos **DML** más comunes:

SELECT. Es posiblemente el comando más utilizado en **SQL**. Permite recuperar datos de una o más tablas de tu base de datos. Aunque estrictamente hablando **SELECT** pertenece al grupo de **DQL (Data Query Language)**, a menudo es incluido como comando de **DLL**. Podemos usar **SELECT** para obtener todos los datos de una tabla, o sólo algunos datos que cumplan con ciertos criterios, por ejemplo:

```
SELECT * FROM Customers;
```

Este comando seleccionará todos los campos (indicado por el asterisco *) de la tabla **Customers**.

También podemos especificar campos individuales en lugar del asterisco, por ejemplo:

```
SELECT FirstName, LastName FROM Customers;
```

INSERT. Este comando se utiliza para insertar nuevos registros en una tabla. Debemos especificar los valores en el mismo orden en que se definieron las columnas en la tabla, por ejemplo:

```
INSERT INTO Customers (FirstName, LastName, Email)  
VALUES ('Juan', 'Perez', 'juan.perez@example.com');
```

En este ejemplo, estamos insertando un nuevo registro en la tabla **Customers**. El nuevo cliente se llama 'Juan Perez' y su correo electrónico es "juan.perez@example.com".

UPDATE. Este comando se utiliza para modificar los datos existentes en una tabla. Podemos cambiar los valores de una o más columnas para uno o más registros, por ejemplo:

```
UPDATE Customers
SET Email = 'juan.p@example.com'
WHERE FirstName = 'Juan' AND LastName = 'Perez';
```

Aquí, estamos cambiando el correo electrónico del cliente 'Juan Perez'. El nuevo correo electrónico es 'juan.p@example.com'. La cláusula WHERE se utiliza para especificar qué registros deben ser actualizados. Si omitimos la cláusula WHERE, todos los registros en la tabla serán actualizados.

DELETE. Este comando se utiliza para eliminar registros de una tabla. De nuevo, debemos tener cuidado al usar este comando, especialmente si omitimos la cláusula **WHERE**.

```
DELETE FROM Customers WHERE Email = 'juan.p@example.com';
```

*En este ejemplo, estamos eliminando el registro del cliente que tiene el correo electrónico 'juan.p@example.com'. Si omite la cláusula **WHERE**, todos los registros en la tabla serán eliminados.*

Cada uno de estos comandos puede tener numerosas variantes y opciones, y pueden combinarse de diversas maneras para realizar consultas más complejas. Además, estos comandos también se pueden combinar con comandos de otros subconjuntos de **SQL**, como los comandos de **DDL** y **DCL**.

2.2.1. Cláusulas y Operadores en SQL

SQL proporciona una serie de cláusulas y operadores que nos permiten crear consultas precisas y flexibles para interactuar con los datos. A continuación, mencionaremos algunos de los más utilizados:

2.2.1.1. Cláusulas en SQL

En **SQL**, las cláusulas son componentes de las instrucciones y consultas. Una cláusula en **SQL** realiza una función específica y es usada para combinar, filtrar y transformar los datos. Las cláusulas pueden ser opcionales y pueden venir en cualquier orden en la consulta, dependiendo de su propósito. Algunas cláusulas comunes incluyen **WHERE**, **GROUP BY**, **HAVING**,

y **ORDER BY**. Las cláusulas también se utilizan en **DQL**. A continuación, veremos las cláusulas más utilizadas.

WHERE. Se utiliza para filtrar los registros que cumplen con una condición específica, por ejemplo:

```
SELECT * FROM Customers WHERE Country = 'Mexico';
```

ORDER BY. Se utiliza para ordenar los registros en el resultado de una consulta **SELECT** en un orden específico (ascendente o descendente), por ejemplo:

```
SELECT * FROM Customers ORDER BY Country ASC;
```

GROUP BY. Se utiliza con los comandos de funciones agregadas (**COUNT**, **MAX**, **MIN**, **AVG**, **SUM**) para agrupar el conjunto de resultados por una o más columnas, por ejemplo:

```
SELECT Country, COUNT(*) FROM Customers GROUP BY Country;
```

HAVING. Es similar a **WHERE**, pero se utiliza con funciones agregadas, por ejemplo:

```
SELECT Country, COUNT(*) FROM Customers GROUP BY Country  
HAVING COUNT(*) > 10;
```

DISTINCT. Se utiliza con **SELECT** para retornar sólo valores distintos, por ejemplo:

```
SELECT DISTINCT Country FROM Customers;
```

JOIN. Se utiliza para combinar filas de dos o más tablas, basado en una relación entre ciertos campos en estas tablas. Hay varios tipos de **JOIN** en **SQL**: **INNER JOIN**, **LEFT JOIN**, **RIGHT JOIN**, y **FULL JOIN**, por ejemplo:

INNER JOIN. Este tipo de **JOIN** devuelve las filas cuando hay una coincidencia en ambas tablas. (Equivale a una operación de intersección entre conjuntos):

```
SELECT Orders.OrderID, Customers.CustomerName  
FROM Orders
```

```
INNER JOIN Customers ON Orders.CustomerID =  
Customers.CustomerID;
```

LEFT JOIN (o LEFT OUTER JOIN). Este tipo de **JOIN** devuelve todas las filas de la tabla de la izquierda, y las filas coincidentes de la tabla de la derecha. Si no hay coincidencias, el resultado es **NULL** del lado derecho. (Equivale a una operación de relación entre conjuntos, donde siempre se prioriza la aparición de elementos en el conjunto izquierdo, aun cuando no tengan correspondencia con elementos del conjunto derecho).

```
SELECT Orders.OrderID, Customers.CustomerName  
FROM Orders  
LEFT JOIN Customers ON Orders.CustomerID =  
Customers.CustomerID;
```

RIGHT JOIN (o RIGHT OUTER JOIN). Este tipo de **JOIN** devuelve todas las filas de la tabla de la derecha, y las filas coincidentes de la tabla de la izquierda. Si no hay coincidencias, el resultado es **NULL** del lado izquierdo. (Equivale a una operación de relación entre conjuntos, donde siempre se prioriza la aparición de elementos en el conjunto derecho, aun cuando no tengan correspondencia con elementos del conjunto izquierdo).

```
SELECT Orders.OrderID, Customers.CustomerName  
FROM Orders  
RIGHT JOIN Customers ON Orders.CustomerID =  
Customers.CustomerID;
```

FULL JOIN (o FULL OUTER JOIN). Este tipo de **JOIN** devuelve las filas cuando hay una coincidencia en una de las tablas. Es decir, devuelve todas las filas de ambas tablas. Si no hay una coincidencia, el resultado es **NULL** en el lado donde no hay coincidencia.

```
SELECT Orders.OrderID, Customers.CustomerName  
FROM Orders  
FULL JOIN Customers ON Orders.CustomerID =  
Customers.CustomerID;
```

Los **JOIN** en **SQL** son herramientas poderosas y flexibles que permiten combinar registros de dos o más tablas en una base de datos relacional. Los ejemplos que hemos discutido hasta ahora son bastante sencillos, pero en la práctica, las consultas con **JOIN** pueden volverse mucho

más complejas y sofisticadas. Podrían implicar múltiples tablas y múltiples condiciones de unión, y podrían incluir una variedad de tipos de **JOIN**.

Los tipos básicos de **JOIN** (**INNER JOIN**, **LEFT JOIN**, **RIGHT JOIN**) son bastante universales, pero no todos los sistemas de bases de datos soportan todos los tipos de **JOIN**. Por ejemplo, **MySQL**, que es uno de los sistemas de gestión de bases de datos más populares, no soporta directamente el **FULL OUTER JOIN**, aunque se la puede emular utilizando una combinación de **LEFT JOIN** y **RIGHT JOIN** junto con la operación **UNION**.

```
SELECT Orders.OrderID, Customers.CustomerName
FROM Orders
LEFT JOIN Customers ON Orders.CustomerID =
Customers.CustomerID;

UNION

SELECT Orders.OrderID, Customers.CustomerName
FROM Orders
RIGHT JOIN Customers ON Orders.CustomerID =
Customers.CustomerID;
```

LIMIT. Se utiliza para especificar el número máximo de registros que debe retornar una consulta, por ejemplo:

```
SELECT * FROM Customers LIMIT 10;
```

2.2.1.2. Operadores en SQL

Los operadores en **SQL** son símbolos que indican una operación a realizar en uno o más operandos. Los operadores permiten al usuario manipular los datos de formas muy específicas. Los operadores pueden ser aritméticos (como **+**, **-**, *****, **/**), de comparación (como **=**, **<>**, **<**, **>**, **<=**, **>=**), lógicos (como **AND**, **OR**, **NOT**) y de concatenación. Por ejemplo, el operador **AND** se utiliza para combinar varias condiciones en una cláusula **WHERE**.

2.2.1.2.1. Operadores Aritméticos

Este tipo de operadores se utiliza para realizar operaciones matemáticas entre valores numéricos.

+ (**suma**). Suma dos números.

```
SELECT 50 + 50;
```

- **(resta)**. Resta el segundo número del primero.

```
SELECT 100 - 50;
```

* **(multiplicación)**. Multiplica dos números.

```
SELECT 10 * 10;
```

/ **(división)**. Divide el primer número por el segundo.

```
SELECT 100 / 10;
```

% **(módulo)**. Devuelve el resto de la división del primer número por el segundo.

```
SELECT 10 % 3;
```

2.2.1.2.2. Operadores de Comparación

Los operadores de comparación comprueban la relación entre dos valores.

= **(igual a)**. Devuelve verdadero si los valores son iguales.

```
SELECT * FROM Customers WHERE Country = 'Germany';
```

<> o != **(diferente de)**. Devuelve verdadero si los valores son diferentes.

```
SELECT * FROM Customers WHERE Country <> 'Germany';
```

< **(menor que)**. Devuelve verdadero si el primer valor es menor que el segundo.

```
SELECT * FROM Products WHERE Price < 10;
```

> **(mayor que)**. Devuelve verdadero si el primer valor es mayor que el segundo.

```
SELECT * FROM Products WHERE Price > 10;
```

<= (menor o igual que). Devuelve verdadero si el primer valor es menor o igual al segundo.

```
SELECT * FROM Products WHERE Price <= 10;
```

>= (mayor o igual que). Devuelve verdadero si el primer valor es mayor o igual al segundo.

```
SELECT * FROM Products WHERE Price >= 10;
```

2.2.1.2.3. Operadores Lógicos.

Sirven para realizar operaciones lógicas, generalmente en condiciones de filtrado.

AND. Devuelve verdadero si ambas condiciones son verdaderas.

```
SELECT * FROM Customers  
WHERE Country = 'Germany'  
AND City = 'Berlin';
```

OR. Devuelve verdadero si al menos una de las condiciones es verdadera.

```
SELECT * FROM Customers  
WHERE Country = 'Germany'  
OR Country = 'USA';
```

NOT. Invierte el valor de verdad de la condición.

```
SELECT * FROM Customers  
WHERE NOT Country = 'Germany';
```

IN. Devuelve verdadero si el valor está dentro de un conjunto especificado.

```
SELECT * FROM Customers  
WHERE Country IN ('Germany', 'USA');
```

BETWEEN. Devuelve verdadero si el valor está dentro de un rango especificado.

```
SELECT * FROM Products WHERE Price BETWEEN 10 AND 20;
```

LIKE. Devuelve verdadero si el valor coincide con un patrón especificado.

```
SELECT * FROM Customers WHERE Country LIKE 'G%';
```

IS NULL. Devuelve verdadero si el valor es NULL.

```
SELECT * FROM Customers WHERE Country IS NULL;
```

En **SQL**, **NULL** es un estado especial que significa que un valor no existe en la base de datos.

2.2.1.2.4. Operadores de Conjunto.

Se utilizan para combinar los resultados de dos o más consultas **SELECT** en un solo conjunto de resultados.

UNION. Devuelve los resultados de varias consultas **SELECT** como un solo conjunto de resultados, eliminando duplicados.

```
SELECT City FROM Customers
UNION
SELECT City FROM Suppliers
ORDER BY City;
```

UNION ALL. Similar a **UNION**, pero mantiene los duplicados.

```
SELECT City FROM Customers
UNION ALL
SELECT City FROM Suppliers
ORDER BY City;
```

INTERSECT. Devuelve los resultados que son comunes a las consultas **SELECT** especificadas.

```
SELECT City FROM Customers
INTERSECT SELECT
City FROM Suppliers;
```

EXCEPT (o MINUS). Devuelve los resultados de la primera consulta **SELECT** que no aparecen en la segunda consulta **SELECT**.

```
SELECT City FROM Customers
EXCEPT SELECT
City FROM Suppliers;
```

2.2.1.2.5. Operadores de Concatenación.

Se utilizan para unir dos o más cadenas en una sola.

|| o + o CONCAT(). une dos o más cadenas en una sola.

```
SELECT FirstName || ' ' || LastName AS FullName FROM
Employees; -- Se usa en Oracle

SELECT FirstName + ' ' + LastName AS FullName FROM
Employees; -- Se usa en SQL Server

SELECT CONCAT(FirstName, ' ', LastName) AS FullName FROM
Employees; -- Se una en MySQL
```

3.3 Lenguaje de Consulta de Datos (DQL)

Mientras que **DML** está diseñado para cambiar los datos, ya sea agregando, modificando o eliminando registros, el **Data Query Language (DQL)** se utiliza para consultar los datos sin cambiarlos. El principal comando **DQL** es **SELECT**, visto anteriormente, que puede estar acompañado de las cláusulas y operadores oportunamente presentadas.

3.4 Lenguaje de Control de Datos (DCL)

El Lenguaje de Control de Datos o **Data Control Language (DCL)** en **SQL** es utilizado para controlar los permisos y el acceso a la base de datos. DCL incluye dos comandos principales: **GRANT** y **REVOKE**.

GRANT. Este comando se utiliza para proporcionar privilegios a los usuarios. Los privilegios determinan las acciones que un usuario puede realizar en la base de datos. Estas acciones pueden incluir **SELECT**, **INSERT**, **UPDATE**, **DELETE**, **EXECUTE** y más. Un ejemplo de cómo se utiliza **GRANT** podría ser el siguiente:


```
GRANT SELECT, INSERT, DELETE ON MyTable TO SomeUser;
```

Este comando daría al usuario **SomeUser** los privilegios para ejecutar comandos **SELECT**, **INSERT** y **DELETE** en la tabla **MyTable**.

REVOKE. Este comando hace exactamente lo contrario que **GRANT**. Se utiliza para quitar los privilegios otorgados a un usuario. Un ejemplo de cómo se utiliza **REVOKE** sería el siguiente:

```
REVOKE SELECT, INSERT, DELETE ON MyTable FROM SomeUser;
```

Este comando eliminaría los privilegios de **SELECT**, **INSERT** y **DELETE** del usuario **SomeUser** en la tabla **MyTable**.

Por lo general, sólo los administradores de base de datos o los usuarios con un alto nivel de privilegios tendrán la capacidad de ejecutar estos comandos.

3.5 Lenguaje de Control de Transacciones (TCL)

El *Transaction Control Language* (TCL) se utiliza para administrar las transacciones en la base de datos. Las transacciones son unidades de trabajo que se pueden confirmar o anular. Estos son los principales comandos del **TCL**:

COMMIT. Este comando se utiliza para guardar todos los cambios realizados desde el último **COMMIT** o **ROLLBACK**. Una vez que se ejecuta un **COMMIT**, todos los cambios son permanentes y no pueden ser revertidos.

```
UPDATE table_name SET column1 = 'value1' WHERE condition;  
COMMIT
```

*En este ejemplo, una vez que se ejecuta el **COMMIT**, los cambios realizados por la consulta de actualización se guardan permanentemente en la base de datos.*

ROLLBACK. Este comando se utiliza para deshacer todas las operaciones desde el último **COMMIT** o **ROLLBACK**.

```
UPDATE table_name SET column1 = 'value1' WHERE condition;  
ROLLBACK;
```

En este ejemplo, la consulta de actualización se deshace y los datos se restauran al estado que tenían en el último **COMMIT** o **ROLLBACK**.

SAVEPOINT. Este comando se utiliza para identificar un punto en una transacción que puede ser usado para revertir parte de la transacción sin revertir toda la transacción. Es una forma de tener un **ROLLBACK** a un punto específico en la transacción en lugar de deshacer toda la transacción.

```
SAVEPOINT savepoint_name;
```

A continuación, si algo va mal con la transacción, podemos hacer un **ROLLBACK** al **SAVEPOINT** en lugar de a toda la transacción, de la siguiente forma:

```
ROLLBACK TO savepoint_name;
```

Esto es útil en situaciones donde se tiene una transacción muy larga y queremos poder revertir parte de ella en caso de que algo salga mal, sin tener que deshacer toda la transacción. Cabe destacar que no todos los sistemas de gestión de bases de datos **SQL** soportan **SAVEPOINT**.

4. Funciones en SQL

En **SQL**, las funciones son procedimientos predefinidos que aceptan parámetros, realizan una acción y luego devuelven el resultado. Las funciones en **SQL** se dividen principalmente en dos tipos: funciones escalares y funciones de conjunto.

4.1 Funciones escalares.

Este tipo de funciones operan en un solo valor y devuelven un solo valor, abarcando una variedad de categorías que incluyen funciones de manipulación de cadenas, funciones matemáticas y funciones de fecha y hora. Algunas funciones escalares comunes en **SQL** incluyen:

LOWER(). Convierte todas las letras de un *string* a minúsculas. Ejemplo:

```
SELECT LOWER('HELLO WORLD') AS LowercaseString;  
-- Resultado: 'hello world'
```

UPPER(). Convierte todas las letras de un *string* a mayúsculas. Ejemplo:

```
SELECT UPPER('hello world') AS UppercaseString;
-- Resultado: 'HELLO WORLD'
```

LEN(). Devuelve la longitud de un *string*. Ejemplo:

```
SELECT LEN('Hello World') AS StringLength;
-- Resultado: 11
```

ROUND(). redondea un número a un número específico de decimales. Ejemplo:

```
SELECT ROUND(123.456, 2) AS RoundedNumber;
-- Resultado: 123.46
```

TRIM(). Elimina espacios en blanco al principio y al final de una cadena. Ejemplo:

```
SELECT TRIM(' Hello World ') AS TrimmedString;
-- Resultado: 'Hello World'
```

NOW(). Esta función devuelve la fecha y hora actuales. Ejemplo:

```
SELECT NOW() AS CurrentDateTime;
```

REPLACE(). Esta función reemplaza todas las ocurrencias de una subcadena especificada dentro de una cadena, por otra subcadena. Ejemplo:

```
SELECT REPLACE('Hello World', 'World', 'SQL') AS Replaced;
-- Resultado: 'Hello World'
```

Estas son sólo algunas de las funciones escalares que existen en **SQL**, recomendamos consultar la documentación del DBMS que se quiera utilizar para más información al respecto.

4.2 Funciones de conjunto.

Este tipo de funciones opera en un conjunto de valores y devuelven un solo valor resumen. Son vitales para el análisis de datos, ya que proporcionan los medios para describir conjuntos de datos. Algunas funciones de conjunto comunes en **SQL** son las siguientes:

AVG(). Calcula la media aritmética de un conjunto de valores. Ejemplo:

```
SELECT AVG(price) AS AveragePrice FROM products;
```

SUM(). Suma todos los valores de un conjunto. Ejemplo:

```
SELECT SUM(quantity) AS TotalQuantity FROM orders;
```

MAX(). Devuelve el valor máximo de un conjunto. Ejemplo:

```
SELECT MAX(age) AS OldestUser FROM users;
```

MIN(). Devuelve el valor mínimo de un conjunto. Ejemplo:

```
SELECT MIN(age) AS YoungestUser FROM users;
```

COUNT(). Cuenta el número de filas que coinciden con un criterio especificado.

```
SELECT COUNT(*) AS TotalUsers FROM users;
```

GROUP_CONCAT(). Esta función concatena los valores de un grupo en un solo *string*, con los valores de cada miembro del grupo separados por un delimitador.

```
SELECT GROUP_CONCAT(name SEPARATOR ', ') AS UserNames FROM users;
```

4.3 Implementación de Funciones en Consultas SQL.

Las funciones en SQL pueden ser utilizadas en diversas partes de una consulta **SQL**. Las funciones escalares pueden ser utilizadas en cualquier parte de una consulta donde se permiten expresiones. Esto incluye la lista de selección, así como las cláusulas **WHERE** y **ORDER BY**. Por ejemplo, el siguiente código seleccionará los nombres de los usuarios cuyos nombres tienen más de 5 caracteres, convertirá esos nombres a mayúsculas y los ordenará por la longitud del nombre:

```
SELECT UPPER(name) AS UpperCaseName, age
FROM users
WHERE LENGTH(name) > 5
ORDER BY LENGTH(name);
```

Por otro lado, las funciones de conjunto se utilizan a menudo en conjunto con la cláusula **GROUP BY** para calcular resúmenes o agregados para cada grupo de valores. Por ejemplo, el siguiente código calcula la edad promedio de los usuarios en cada ciudad:

```
SELECT city, AVG(age) AS AverageAge
FROM users
GROUP BY city;
```

5. Store Procedures

Los Procedimientos Almacenados (**SP**, por sus siglas en inglés) en **SQL** son una característica esencial que permite a los desarrolladores y administradores de bases de datos crear rutinas eficientes, reutilizables y modulares para realizar operaciones en la base de datos. Son fundamentales para mantener la coherencia de los datos, encapsular la lógica de negocios y optimizar las consultas complejas. Estas potentes rutinas, escritas en **SQL** y almacenadas en la propia base de datos, pueden ser invocadas por una aplicación, un usuario o incluso otros objetos de la base de datos, como los *triggers*.

Al precompilarse y permitir parámetros, los procedimientos almacenados ofrecen ventajas significativas en términos de rendimiento y flexibilidad. A lo largo de este tema, exploraremos en detalle los procedimientos almacenados, su sintaxis, cómo se crean, cómo se

invocan y cómo pueden mejorar la gestión de sus bases de datos. Los **SP** Ofrecen varias ventajas importantes:

Precompilación. Los **SP** son precompilados, lo que significa que la base de datos ya sabe cómo va a ejecutarlos, ahorrando tiempo durante la ejecución.

Invocación. Pueden ser invocados de manera explícita, lo que proporciona un control preciso sobre cuándo se ejecutan. También pueden ser llamados por otros objetos de la base de datos, como *triggers*.

Parametrización. Los *Stored Procedures* permiten el uso de parámetros. Esto significa que podemos reutilizar el mismo código con diferentes datos, aumentando la eficiencia y la flexibilidad.

Operaciones CRUD. Son capaces de realizar operaciones de lectura y escritura en la base de datos. Esto incluye operaciones de Creación, Lectura, Actualización y Eliminación (**CRUD**).

Encapsulamiento de lógica de negocios. Permiten encapsular la lógica de negocios y las reglas en la base de datos, manteniendo la coherencia de los datos y facilitando su mantenimiento.

Los procedimientos almacenados son una herramienta poderosa para hacer nuestro código **SQL** más reutilizable, modular y eficiente. Sin embargo, también debemos ser conscientes de que, si se usan incorrectamente, pueden introducir complejidad y problemas de rendimiento. La sintaxis para crear un procedimiento almacenado puede variar ligeramente dependiendo del **DBMS** que estés utilizando. A continuación, se muestra un ejemplo general utilizando la sintaxis de **MySQL**:

```
CREATE PROCEDURE procedure_name (parameter1 datatype,  
parameter2 datatype, ...)  
BEGIN  
-- SQL statements  
END;
```

En el caso anterior, *procedure_name* es el nombre del procedimiento almacenado, *parameter1*, *parameter2*, ... son los parámetros que puedes pasar al procedimiento almacenado, y **BEGIN ... END** contiene las declaraciones SQL que se ejecutarán cuando se llame al procedimiento almacenado.

Por ejemplo, podríamos tener un procedimiento almacenado para seleccionar todos los registros de una tabla **employees** que tienen un salario superior a un valor dado:

```
CREATE PROCEDURE GetHighEarners(IN min_salary INT)
BEGIN
    SELECT * FROM employees WHERE salary > min_salary;
END;
```

En este ejemplo, **GetHighEarners** es el nombre del procedimiento almacenado y **min_salary** es un parámetro que pasamos al procedimiento almacenado. Cuando llamamos a este procedimiento almacenado, nos devolverá todos los empleados que ganen más que el salario mínimo proporcionado. Para llamar a este procedimiento almacenado, podríamos usar la siguiente sintaxis:

```
CALL GetHighEarners(50000);
```

Esto nos devolvería todos los empleados que ganan más de 50,000.

6. Triggers en SQL

Los **triggers** o disparadores en **SQL** son procedimientos almacenados que se ejecutan automáticamente en respuesta a ciertos eventos en una tabla en una base de datos. Los eventos que los pueden activar son generalmente son las operaciones de inserción, actualización o eliminación. Los **triggers** tienen tres componentes clave:

Evento. Este es el evento que activará el **trigger**. Puede ser una operación de inserción, actualización o eliminación.

Tiempo. El trigger puede definirse para que se ejecute antes o después del evento. Los **triggers BEFORE** se utilizan para validar o cambiar los datos antes de que se escriban en la tabla. Los **triggers AFTER** se utilizan para registrar cambios o realizar acciones en función de los cambios, como actualizar una tabla relacionada.

Acción. Es el conjunto de instrucciones **SQL** que se ejecutarán cuando se dispare el trigger.

Un ejemplo de la sintaxis para crear un trigger en **SQL** es el siguiente:

```
CREATE TRIGGER trigger_name
BEFORE INSERT ON table_name
FOR EACH ROW
BEGIN
    -- SQL statements
END;
```

Aquí, **trigger_name** es el nombre del **trigger**, **INSERT** es el evento que activará el **trigger**, **table_name** es la tabla donde se aplica el trigger, y **BEGIN ... END** contiene las declaraciones SQL que se ejecutarán cuando se dispare el trigger.

Un **trigger AFTER UPDATE** se utiliza para realizar una acción después de que una operación de actualización haya ocurrido en una tabla. Por ejemplo, imaginemos que tenemos una tabla de **employees** y una tabla **audit_log**. Queremos registrar en **audit_log** cada vez que se cambie el salario de un empleado.

```
CREATE TRIGGER salary_change_log
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    IF NEW.salary <> OLD.salary THEN
        INSERT INTO audit_log(employee_id, old_salary,
                               new_salary, change_date)
        VALUES(NEW.id, OLD.salary, NEW.salary, NOW());
    END IF;
END;
```

En el ejemplo anterior, **salary_change_log** es el nombre del **trigger**, **UPDATE** es el evento que activará el **trigger**, y **employees** es la tabla donde se aplica el **trigger**.

Cuando se dispare este **trigger** (es decir, cuando el salario de un empleado se actualiza), verificará si el nuevo salario (**NEW.salary**) es diferente del antiguo salario (**OLD.salary**). Si es así, entonces insertará una nueva fila en la tabla **audit_log** con el ID del empleado, el antiguo salario, el nuevo salario y la fecha y hora actuales (**NOW()**).

Debemos recordar que los **triggers** deben usarse con precaución, ya que pueden provocar efectos secundarios inesperados si no se manejan correctamente. Por ejemplo, un **trigger** puede activar otro **trigger**, lo que puede llevar a una cadena de eventos difícil de seguir y controlar.

Por otro lado, los **triggers** pueden ralentizar las operaciones de inserción, actualización y eliminación, ya que el sistema de base de datos necesita tiempo para procesar los **triggers** además de la operación original.

7. Estructuras de Control en SQL.

Las estructuras de control en **SQL** son un conjunto de instrucciones que permiten manipular el flujo de ejecución de los bloques de código **SQL**, proporcionando así una mayor flexibilidad y funcionalidad a los desarrolladores de bases de datos. Al igual que con otros lenguajes de programación, las estructuras de control en **SQL** pueden utilizarse para implementar lógica condicional, realizar tareas repetitivas y alterar la secuencia normal de ejecución del código, dependiendo de ciertas condiciones.

Estas estructuras de control se utilizan comúnmente en el contexto de procedimientos almacenados, funciones y **triggers**. Es importante tener en cuenta que la sintaxis y las capacidades exactas de estas estructuras de control pueden variar dependiendo del **DBMS** que estemos utilizando.

SQL, como lenguaje, sigue una serie de normas establecidas, pero es importante reconocer que los diferentes **DBMS** tienen la capacidad de implementar y expandir estas normas de formas únicas y diversas. En consecuencia, siempre es aconsejable que los desarrolladores consulten la documentación de su **DBMS** específico para comprender completamente cómo las estructuras de control están implementadas y cómo pueden ser más efectivamente utilizadas.

Además, aunque las estructuras de control pueden añadir mucha potencia y flexibilidad a tu código **SQL**, también pueden añadir complejidad. Por lo tanto, es importante usarlas con cuidado y asegurarnos de que nuestro código sigue siendo fácil de entender y mantener.

A continuación, daremos una descripción de las estructuras de control más comunes en **SQL**:

IF-THEN-ELSE. Este es un condicional básico que ejecuta un bloque de código si una condición es verdadera y otro bloque de código si la condición es falsa. También puede contener múltiples condiciones utilizando **ELSEIF**.

```
IF user_age > 18 THEN
  -- Esta parte del código se ejecuta si el usuario es mayor de 18
  INSERT INTO adults (user_id) VALUES (user_id);
ELSE
  -- Esta parte del código se ejecuta si el usuario es 18 o menor
  INSERT INTO minors (user_id) VALUES (user_id);
END IF;
```

CASE. Esta estructura de control permite a los programadores elegir entre múltiples bloques de código basándose en el valor de una expresión. Es similar a usar múltiples declaraciones **IF-ELSE**, pero puede ser más conciso y legible.

```
CASE user_role

WHEN 'admin' THEN
  -- Esta parte del código se ejecuta si el rol del usuario es 'admin'
  UPDATE users SET permissions = 'all';

WHEN 'user' THEN
  -- Esta parte se ejecuta si el rol del usuario es 'user'
  UPDATE users SET permissions = 'limited';

ELSE
  -- Esta parte del código se ejecuta si el rol del usuario no es 'admin' ni 'user'
  UPDATE users SET permissions = 'none';

END CASE;
```

LOOP (WHILE). Un **loop** permite repetir un bloque de código varias veces. Hay varios tipos de **loops** en **SQL**, incluyendo los **WHILE loops**, que repiten un bloque de código mientras una condición sea verdadera, y los **REPEAT loops**, que repiten un bloque de código hasta que una condición se vuelva verdadera.

```
DECLARE @counter INT;
SET @counter = 1;

WHILE @counter <= 10
BEGIN
    -- Esta parte del código se ejecuta 10 veces
    PRINT @counter;
    SET @counter = @counter + 1;
END;
```

LEAVE. Esta estructura de control permite a los programadores salir de un **loop** antes de que la condición del **loop** se vuelva falsa. Es similar a la declaración **BREAK** en muchos otros lenguajes de programación. (En **MySQL** se usa un bucle **REPEAT** y la instrucción **LEAVE** para salir del bucle cuando se cumple la condición):

```
SET @counter = 0;

REPEAT
    -- Incrementa el contador
    SET @counter = @counter + 1;

    -- Si el contador llega a 5, sal del bucle
    IF @counter = 5 THEN
        LEAVE;
    END IF;
UNTIL @counter > 10
END REPEAT;
```

ITERATE. Este comando permite a los programadores saltar a la próxima iteración de un **loop**, ignorando el resto del código en el bloque del **loop** actual. Es similar a la declaración **CONTINUE** en muchos otros lenguajes de programación. (En MySQL se usa un bucle **REPEAT** y la instrucción **ITERATE** para saltar a la siguiente iteración del bucle cuando se cumple la condición):

```
SET @counter = 0;
my_loop: REPEAT
  -- Incrementa el contador
  SET @counter = @counter + 1;

  -- Si el contador es 5, salta a la sig. iteración del bucle
  IF @counter = 5 THEN
    ITERATE my_loop;
  END IF;

  -- Esta parte solo se ejecutará si el contador no es 5
  INSERT INTO my_table (counter) VALUES (@counter);
UNTIL @counter > 10
END REPEAT;
```

Los ejemplos anteriores son simplificados y pueden no funcionar en todos los sistemas de gestión de bases de datos debido a las diferencias en la sintaxis **SQL**. Resulta fundamental revisar la documentación de nuestro **DBMS** específico para asegurarnos de que estás utilizando la sintaxis correcta.

8. Vistas en SQL

Una vista en **SQL** es una tabla virtual basada en el resultado de una consulta **SQL**. No almacena datos físicamente, sino que los datos existen en una o más tablas referenciadas en la consulta. Una vista contiene filas y columnas, al igual que una tabla real. Las columnas de una vista son las columnas de las tablas de la consulta.

8.1 Ventajas de las vistas en SQL.

La utilización de vistas tiene una serie de ventajas que detallaremos a continuación:

Seguridad. Las vistas pueden mostrar columnas específicas de la tabla, lo que puede ser útil si no queremos que ciertos usuarios vean ciertas columnas.

Reducción de la Complejidad. Si tenemos una consulta SQL que es muy complicada, podemos guardarla como una vista y luego referenciar la vista, lo que puede simplificar nuestro código.

Reutilización de código. Si varias consultas SQL necesitan los mismos JOINS, cálculos, etc., podemos crear una vista y reutilizarla en todas las consultas, lo que puede ser más eficiente.

8.2 Sintaxis en la creación de vistas.

La sintaxis para la creación de vistas es la siguiente:

```
CREATE VIEW view_name AS SELECT column1, column2, ... FROM
table_name WHERE [condition];
```

Por ejemplo, si tenemos una tabla de empleados y solo queremos que ciertos usuarios vean los nombres y salarios de los empleados, podemos crear una vista como esta:

```
CREATE VIEW employee_view AS SELECT name, salary FROM employee;
```

Luego podemos seleccionar datos de la vista así:

```
SELECT * FROM employee_view;
```

Es importante recordar que las vistas se actualizan dinámicamente. Si los datos en las tablas subyacentes cambian, las vistas que se basan en esas tablas también cambiarán para reflejar los nuevos datos.

8.3 Tipos de vistas en SQL

Las vistas en SQL se pueden clasificar en dos categorías principales: vistas actualizables y vistas de solo lectura.

Vistas actualizables. Una vista actualizable permite que los cambios se realicen en los datos mediante la vista. Es decir, puedes realizar operaciones de inserción, actualización y eliminación a través de la vista, y estos cambios se reflejarán en la tabla de base de datos subyacente. No todas las vistas son actualizables. Para que una vista sea actualizable, debe cumplir ciertas

restricciones. Por ejemplo, la vista debe incluir solo una tabla base y no debe contener funciones de agregación, cláusulas DISTINCT, etc.

Vistas de solo lectura. Estas vistas no permiten realizar cambios en los datos a través de ellas. Las operaciones de actualización, inserción y eliminación no están permitidas. Este tipo de vistas se crea cuando se utiliza una función de agregación, la cláusula DISTINCT, UNION, etc.

8.4 Vistas materializadas.

Las vistas materializadas son una forma especial de vistas que se utilizan para mejorar el rendimiento de la consulta, especialmente en bases de datos grandes y complejas donde las consultas de agregación y las uniones son costosas en términos de tiempo de procesamiento.

En una vista normal, la consulta que define la vista se ejecuta cada vez que se accede a la vista. Por ejemplo, si tienes una vista que suma los totales de ventas de una tienda, cada vez que consultas esa vista, SQL realizará la suma de todos los datos relevantes en las tablas subyacentes.

En cambio, una vista materializada no ejecuta la consulta cada vez que se accede a ella. En lugar de eso, cuando se crea la vista materializada, se ejecuta la consulta y se almacena el resultado como una tabla física en el disco. Cuando consultamos la vista materializada, simplemente obtenemos los datos almacenados, en lugar de calcularlos nuevamente. Esto puede ser mucho más rápido para consultas complejas.

La desventaja de las vistas materializadas es que, dado que almacenan los resultados de la consulta, deben actualizarse cada vez que los datos de las tablas subyacentes cambien. Dependiendo de la base de datos y la configuración, esto puede suceder automáticamente, o puede que necesites actualizar la vista materializada manualmente. Esta actualización puede ser costosa en términos de tiempo de procesamiento y espacio en disco, por lo que debemos balancear el costo de la actualización con el beneficio del rápido tiempo de consulta.

Las vistas materializadas son compatibles con algunos sistemas de gestión de bases de datos, como PostgreSQL y Oracle. Sin embargo, a partir de julio de 2023, MySQL aún no admite vistas materializadas de manera nativa. Aun así, hay formas de emular este comportamiento en

MySQL, aunque no son tan convenientes ni eficientes como el soporte nativo para las vistas materializadas.

9. Subconsultas en SQL.

Uno de los conceptos más poderosos en **SQL** es la denominada subconsulta, también conocida como consulta interna o consulta anidada. Una subconsulta es esencialmente una consulta dentro de otra consulta, y es utilizada para simplificar consultas complejas al dividir las en partes manejables.

Las subconsultas permiten a los usuarios manipular los datos de manera más eficiente, realizando operaciones que de otro modo serían difíciles o imposibles. Nos permiten seleccionar datos basándonos en el resultado de otra consulta. En otras palabras, las subconsultas nos brindan la capacidad de realizar consultas basadas en un conjunto de resultados obtenido de otra consulta.

Las subconsultas pueden devolver una sola fila, múltiples filas, una única columna o múltiples columnas, y también pueden ser correlacionadas, lo que significa que dependen de datos de la consulta exterior. Cada una de estas variantes tiene sus propias aplicaciones y su utilización dependerá de las necesidades específicas del usuario.

A continuación, mostramos un ejemplo simple de cómo se puede usar una subconsulta en SQL:

```
SELECT nombre
FROM empleados
WHERE salario > (SELECT AVG(salario) FROM empleados);
```

*En este ejemplo, la subconsulta **SELECT AVG(salario) FROM empleados** calcula el salario promedio de todos los empleados. Luego, la consulta principal utiliza este valor para seleccionar solo los empleados que ganan más que el salario promedio.*

Existen varios tipos de subconsultas:

Subconsultas de una sola fila. Devuelven una sola fila de la tabla.

```
SELECT nombre, salario
FROM empleados
WHERE salario > (SELECT salario FROM empleados WHERE nombre =
'Juan');
```

Subconsultas de varias filas. Devuelven varias filas de la tabla.

```
SELECT nombre, salario
FROM empleados
WHERE salario IN (SELECT salario FROM empleados WHERE
departamento_id = 10);
```

Subconsultas de varias columnas. Devuelven varias columnas de la tabla.

```
SELECT nombre, salario
FROM empleados
WHERE (departamento_id, salario) IN
(
SELECT departamento_id, AVG(salario)
FROM empleados
GROUP BY departamento_id
);
```

Subconsultas correlacionadas. Son subconsultas que dependen de la consulta exterior.

```
SELECT e1.nombre, e1.salario
FROM empleados e1
WHERE salario >
(
SELECT AVG(salario)
FROM empleados e2
WHERE e1.departamento_id = e2.departamento_id
);
```

En esta consulta, cada fila de la tabla exterior **empleados e1** se compara con todas las filas de la tabla interior **empleados e2**.

BLOQUE PRACTICO

10. Ejercicios Resueltos.

A continuación, proponemos una serie de 45 ejercicios con sus respectivas respuestas. Además de los conocimientos teóricos, resulta fundamental la práctica para alcanzar un nivel adecuado de SQL. *Atención: Suponemos que la Base de datos ha sido creada.*

10.1 Creación de Bases de Datos y Tablas en SQL

1. Ejercicio: Crea una tabla llamada "Estudiantes" con las columnas "ID_Estudiante" (entero), "Nombre" (texto), "Edad" (entero), y "Grado" (texto).

Solución:

```
CREATE TABLE Estudiantes (ID_Estudiante INT, Nombre  
VARCHAR(100), Edad INT, Grado VARCHAR(50) );
```

2. Ejercicio: Inserta 3 registros en la tabla "Estudiantes".

Solución:

```
INSERT INTO Estudiantes (ID_Estudiante, Nombre, Edad,  
Grado) VALUES (1, 'Juan', 12, 'Séptimo'), (2, 'María',  
13, 'Octavo'), (3, 'Pedro', 14, 'Noveno');
```

3. Ejercicio: Selecciona todos los registros de la tabla "Estudiantes".

Solución:

```
SELECT * FROM Estudiantes;
```

4. Ejercicio: Actualiza el "Grado" del estudiante con "ID_Estudiante" igual a 2 a 'Noveno'.

Solución:

```
UPDATE Estudiantes SET Grado = 'Noveno' WHERE  
ID_Estudiante = 2;
```

5. Ejercicio: Elimina el registro del estudiante con "ID_Estudiante" igual a 1.

Solución:

```
DELETE FROM Estudiantes WHERE ID_Estudiante = 1;
```

10.2 Select, From, Where, Order By

6. Ejercicio: Selecciona sólo el "Nombre" y "Grado" de todos los estudiantes.

Solución:

```
SELECT Nombre, Grado FROM Estudiantes;
```

7. Ejercicio: Selecciona los estudiantes con "Edad" mayor que 12.

Solución:

```
SELECT * FROM Estudiantes WHERE Edad > 12;
```

8. Ejercicio: Selecciona los estudiantes en 'Octavo' grado.

Solución:

```
SELECT * FROM Estudiantes WHERE Grado = 'Octavo';
```

9. Ejercicio: Selecciona todos los estudiantes y ordena los resultados por "Edad" en orden descendente.

Solución:

```
SELECT * FROM Estudiantes ORDER BY Edad DESC;
```

10. Ejercicio: Selecciona los estudiantes con "Edad" mayor que 12 y ordena los resultados por "Grado" en orden ascendente.

Solución:

```
SELECT * FROM Estudiantes WHERE Edad > 12 ORDER BY Grado ASC;
```

10.3 Group By, Having, Count, Avg, Sum, Min, Max

11.Ejercicio: Encuentra la edad promedio de los estudiantes.

Solución:

```
SELECT AVG(Edad) FROM Estudiantes;
```

12.Ejercicio: Encuentra el número total de estudiantes.

Solución:

```
SELECT COUNT(*) FROM Estudiantes;
```

13.Ejercicio: Encuentra la edad mínima y máxima de los estudiantes.

Solución:

```
SELECT MIN(Edad), MAX(Edad) FROM Estudiantes;
```

14.Ejercicio: Encuentra el número total de estudiantes en cada grado.

Solución:

```
SELECT Grado, COUNT(*) FROM Estudiantes GROUP BY Grado;
```

15.Ejercicio: Encuentra el grado con más de 1 estudiante.

Solución:

```
SELECT Grado, COUNT(*) FROM Estudiantes  
GROUP BY Grado HAVING COUNT(*) > 1;
```

10.4 Join, Inner Join, Left Join, Right Join, Full Join, Cross Join

16.Ejercicio: Crea una tabla "Profesores" con las columnas "ID_Profesor" (entero), "Nombre" (texto), y "Materia" (texto).

Solución:

```
CREATE TABLE Profesores (ID_Profesor INT, Nombre  
VARCHAR(100), Materia VARCHAR(50) );
```

17.Ejercicio: Inserta 3 registros en la tabla "Profesores".

Solución:

```
INSERT INTO Profesores (ID_Profesor, Nombre, Materia)  
VALUES (1, 'Carlos', 'Matemáticas'),  
2, 'Ana', 'Inglés'), (3, 'Roberto', 'Ciencias');
```

18.Ejercicio: Añade una columna "ID_Profesor" a la tabla "Estudiantes".

Solución:

```
ALTER TABLE Estudiantes ADD COLUMN ID_Profesor INT;
```

19.Ejercicio: Actualiza la tabla "Estudiantes" para que cada estudiante tenga un profesor.

Solución:

```
UPDATE Estudiantes SET ID_Profesor = 1 WHERE ID_Estudiante = 1;  
UPDATE Estudiantes SET  
ID_Profesor = 2 WHERE ID_Estudiante = 2; UPDATE Estudiantes SET  
ID_Profesor = 3 WHERE  
ID_Estudiante = 3;
```

20.Ejercicio: Usa INNER JOIN para seleccionar todos los estudiantes y sus respectivos profesores.

Solución:

```
SELECT Estudiantes.Nombre AS Nombre_Estudiante,  
Profesores.Nombre AS Nombre_Profesor FROM Estudiantes  
INNER JOIN Profesores ON Estudiantes.ID_Profesor =  
Profesores.ID_Profesor;
```

10.5 Union, Intersect, Except

21.Ejercicio: Crea una segunda tabla "Estudiantes2" con la misma estructura que "Estudiantes" e inserta algunos registros en ella.

Solución:

```
CREATE TABLE Estudiantes2 (ID_Estudiante INT, Nombre
VARCHAR(100), Edad INT, Grado VARCHAR(50), ID_Profesor INT );

INSERT INTO Estudiantes2 (ID_Estudiante, Nombre, Edad, Grado,
ID_Profesor)

VALUES (4, 'Lucas', 12, 'Séptimo', 1), (5, 'Sofía', 13,
'Octavo', 2), (6, 'Marta', 14, 'Noveno', 3);
```

22.Ejercicio: Usa UNION para seleccionar todos los estudiantes de ambas tablas.

Solución:

```
SELECT * FROM Estudiantes UNION SELECT * FROM Estudiantes2;
```

23.Ejercicio: Usa INTERSECT para seleccionar los estudiantes que aparecen en ambas tablas.

Solución:

```
SELECT * FROM Estudiantes INTERSECT SELECT * FROM Estudiantes2;
```

24.Ejercicio: Usa EXCEPT para seleccionar los estudiantes que están en "Estudiantes" pero no en "Estudiantes2".

Solución:

```
SELECT * FROM Estudiantes EXCEPT SELECT * FROM Estudiantes2;
```

25.Ejercicio: Usa UNION ALL para seleccionar todos los estudiantes de ambas tablas, incluyendo duplicados.

Solución:

```
SELECT * FROM Estudiantes UNION ALL SELECT * FROM Estudiantes2;
```

10.6 Subconsultas, Any, All, Exists

26.Ejercicio: Usa una subconsulta para seleccionar los estudiantes que son más jóvenes que el estudiante más viejo.

Solución:

```
SELECT * FROM Estudiantes WHERE Edad < (SELECT MAX(Edad)
FROM estudiantes);
```

27.Ejercicio: Usa ANY para seleccionar los profesores que enseñan a más de un estudiante.

Solución:

```
SELECT * FROM Profesores
WHERE ID_Profesor = ANY (SELECT ID_Profesor FROM Estudiantes
GROUP BY ID_Profesor HAVING COUNT(ID_Estudiante) > 1);
```

28.Ejercicio: Usa ALL para seleccionar los profesores que enseñan a todos los estudiantes.

Solución:

```
SELECT * FROM Profesores
WHERE ID_Profesor = ALL (SELECT ID_Profesor FROM
Estudiantes);
```

29.Ejercicio: Usa EXISTS para seleccionar los profesores que tienen al menos un estudiante.

Solución:

```
SELECT * FROM Profesores WHERE EXISTS (SELECT 1 FROM
Estudiantes WHERE Estudiantes.ID_Profesor =
Profesores.ID_Profesor);
```

30.Ejercicio: Usa una subconsulta en el FROM para seleccionar el número de estudiantes por profesor.

Solución:

```
SELECT ID_Profesor, COUNT(*) AS Num_Estudiantes FROM (SELECT
ID_Profesor FROM Estudiantes) AS Estudiantes_Profesor GROUP
BY ID_Profesor;
```


10.7 Crear, Alterar, Eliminar Tablas

31.Ejercicio: Crea una tabla "Cursos" con las columnas "ID_Curso" (entero), "Nombre" (texto), y "Créditos" (entero).

Solución:

```
CREATE TABLE Cursos (ID_Curso INT, Nombre VARCHAR(100),  
Credito INT);
```

32.Ejercicio: Añade una columna "ID_Curso" a la tabla "Estudiantes".

Solución:

```
ALTER TABLE Estudiantes ADD COLUMN ID_Curso INT;
```

33.Ejercicio: Cambia el tipo de la columna "Créditos" en la tabla "Cursos" a DECIMAL.

Solución:

```
ALTER TABLE Cursos ALTER COLUMN Credito DECIMAL;
```

34.Ejercicio: Elimina la columna "Créditos" de la tabla "Cursos".

Solución:

```
ALTER TABLE Cursos DROP COLUMN Credito;
```

35.Ejercicio: Elimina la tabla "Cursos".

Solución:

```
DROP TABLE Cursos;
```

10.8 Insert, Update, Delete

36.Ejercicio: Inserta un registro en la tabla "Estudiantes".

Solución:

```
INSERT INTO Estudiantes (ID_Estudiante, Nombre, Edad, Grado, ID_Profesor) VALUES (7, 'Andres', 12, 'Séptimo', 1);
```

37.Ejercicio: Actualiza la "Edad" del estudiante con "ID_Estudiante" igual a 7 a 13.

Solución:

```
UPDATE Estudiantes SET Edad = 13 WHERE ID_Estudiante = 7;
```

38.Ejercicio: Elimina el registro del estudiante con "ID_Estudiante" igual a 7.

Solución:

```
DELETE FROM Estudiantes WHERE ID_Estudiante = 7;
```

39.Ejercicio: Inserta varios registros en la tabla "Estudiantes" usando una sola consulta.

Solución:

```
INSERT INTO Estudiantes  
(ID_Estudiante, Nombre, Edad, Grado, ID_Profesor)  
VALUES  
(8, 'Fernando', 12, 'Séptimo', 1),  
(9, 'Carla', 13, 'Octavo', 2), (10, 'Ines', 14, 'Noveno', 3);
```

40.Ejercicio: Actualiza la "Edad" de todos los estudiantes a 15.

Solución:

```
UPDATE Estudiantes SET Edad = 15;
```

10.9 Funciones, Alias, Concatenación

41.Ejercicio: Selecciona el nombre en mayúsculas de todos los estudiantes.

Solución:

```
SELECT UPPER(Nombre) FROM Estudiantes;
```

42.Ejercicio: Selecciona la longitud del nombre de todos los estudiantes.

Solución:

```
SELECT LENGTH(Nombre) FROM Estudiantes;
```

43.Ejercicio: Selecciona los estudiantes cuyo nombre comienza con 'A'.

Solución:

```
SELECT * FROM Estudiantes WHERE Nombre LIKE 'A%';
```

44.Ejercicio: Selecciona la edad media de los estudiantes.

Solución:

```
SELECT AVG(Edad) FROM Estudiantes;
```

45.Ejercicio: Selecciona la concatenación del nombre y el grado de todos los estudiantes, y da un alias a la columna resultante.

Solución:

```
SELECT CONCAT(Nombre, ' - ', Grado) AS 'Nombre y Grado' FROM Estudiantes;
```


Conclusión.

Al finalizar este trabajo de aproximación al lenguaje SQL, desde el Centro de Estudios en Modelos de Inteligencia Artificial aplicados a las Ciencias Económicas de la Universidad de Buenos Aires (CeMIACE), invitamos a reflexionar sobre la relevancia y la versatilidad de este poderoso lenguaje de consulta estructurado. Apenas hemos arañado la superficie de su poder, explorando diversas facetas de SQL, desde la realización de consultas simples hasta el dominio de subconsultas más avanzadas y la implementación de estructuras de control.

SQL es un lenguaje dinámico que, aunque sigue ciertas normas, puede ser implementado y extendido de diversas formas dependiendo del Sistema de Gestión de Bases de Datos que estemos utilizando. La habilidad de utilizar SQL de manera eficiente y efectiva puede marcar la diferencia entre simplemente tener acceso a una base de datos y ser capaz de extraer, manipular y entender profundamente los datos que contiene. A través de la correcta aplicación de consultas, subconsultas y estructuras de control, podemos responder a preguntas complejas y proporcionar información valiosa que de otro modo sería difícil de obtener.

No obstante, con el poder también viene la responsabilidad. Es esencial usar SQL con precaución y ética, para garantizar que nuestro código sea eficiente, fácil de mantener y respetuoso con la seguridad y privacidad de los datos.

Recordemos también que el aprendizaje de SQL es un camino continuo. Las necesidades de la gestión de datos y las capacidades de los DBMS siguen evolucionando, y los buenos desarrolladores de bases de datos también deben hacerlo. A medida que adquirimos más experiencia y nos enfrentamos a problemas más complejos, seguiremos descubriendo nuevas formas de aplicar las herramientas que SQL nos proporciona.

SQL es una herramienta esencial para cualquier persona que trabaje con bases de datos. Esperamos que este trabajo haya proporcionado a los lectores una base sólida sobre la que se pueda seguir construyendo habilidades y conocimientos en SQL, y por qué no, una fuente de motivación para sacar el máximo provecho de este increíble lenguaje de consulta.

¡Feliz codificación!

Breve Introducción al lenguaje SQL

En nuestra era actual, caracterizada por la explosión de datos, el Lenguaje de Consulta Estructurada, o SQL, ha tomado una importancia vital en una variedad de campos. Como principal herramienta utilizada para interactuar con las bases de datos, SQL es una habilidad esencial para cualquiera que busque analizar, manipular y comprender datos. Este trabajo, "Breve Introducción al Lenguaje SQL", proporciona una visión compacta pero comprensiva de los aspectos más fundamentales de SQL.

